Clock Robustness and Alarm Precision in LLM-Generated and Upstream Zephyr RTC Tests

Pranav Krishnan
The University of Texas at Austin
Austin, Texas, USA

Keshav Bhargava
The University of Texas at Austin
Austin, Texas, USA

ABSTRACT

Firmware tests often wait a fixed time for a hardware event. If a chip's clock runs slower than expected but remains within its datasheet limits, a correct program can fail its test. We compared selected human-written upstream Zephyr real-time clock (RTC) tests and functionally eligible LLM-generated suites under legal oscillator settings and separate alarm faults. The three upstream cases failed at 17–26 of 100 sampled legal settings; all 37 measured LLM failures came from three pending suites generated with the base prompt. The suite-level contrast, 3/24 with the base prompt versus 0/26 with clock ranges, was not significant. In a task-specific alarm-offset probe, evaluable generated callback suites detected 18/18 early and 14/18 late faults, while one upstream callback case detected neither; the LLM specification explicitly requested configured-time firing. Clock robustness, generation yield, and oracle precision require separate assessment.

KEYWORDS

embedded testing, flaky tests, interrupt-driven firmware, RTC, large language models

1 INTRODUCTION

A real-time clock (RTC) alarm test may schedule an event ten calendar seconds ahead, wait thirteen host seconds, and assert that a callback arrived. The assertion is valid only if thirteen seconds accommodates the RTC clock and delivery path. Repeating the same simulator schedule cannot test that premise. Changing the oscillator within its documented range can change the verdict without changing test or firmware source.

We call the selected human-written Zephyr cases *upstream tests*. We compare their clock sensitivity with LLM-generated tests and then challenge surviving tests with faults that separate clock tolerance from alarm-oracle precision. The upstream cases were chosen for visible timing windows, not sampled to represent Zephyr generally. We ask: **RQ1**, which tests flip under legal clock settings? **RQ2**, which faults do they detect? **RQ3**, how accurately does a static threshold predict flips, and for which tests? **RQ4**, which source-level timing assumptions remain? We contribute a frozen paired clock schedule, prospective threshold validation, replay and reference triage, all-attempt generation accounting, and task-matched fault probes. Sampled-setting shares below are simulation sensitivities, not field failure probabilities.

2 APPROACH

Clock model. The pinned Zephyr revision [19] supplies upstream alarm-callback, alarm-pending, and calendar-counting cases. The STM32F4 Discovery board definition selects the low-speed internal oscillator (LSI) as the RTC source. At 3 V across −40 to 105 °C, the STM32F407 datasheet characterizes LSI at 17–47 kHz [14]. With the build's fixed 128×250 prescaler, ten RTC seconds ideally take $t(f) = 320{,}000/f$ host seconds: 6.809 s at 47 kHz and 18.824 s at 17 kHz. The input-clock range does not bound interrupt-delivery latency.

Threshold forecast. For tests with a scalar observation window, the ideal model predicts a failure when $320{,}000/f$ exceeds that window. Forecasts were frozen before clock-variation runs; tests whose callback ordering or polling prevents a scalar bound have no such prediction.

Fault probes. We test out-of-spec fast and slow oscillator inputs, a nominal-frequency *nonarrival fault* that suppresses callback/pending delivery or freezes counting reads, and separate nominal-clock driver faults that shift the alarm compare by −1 or +1 RTC second. A *kill* requires an activated fault and an explicit failure of the selected test case.

3 EXPERIMENTAL SETUP

Models, prompts, and gates. The model IDs `gpt-6-luna`, `gpt-6-sol`, and `gpt-6-astra` (Luna, Sol, and Astra below) each returned five independent, source-only C tests using ztest (Zephyr's test framework) for 13 tasks under two prompt conditions: 390 generations. The *base prompt* omits numeric oscillator limits; the *prompt with clock ranges* adds low-speed and high-speed internal oscillator (HSI) ranges plus a callback-delivery reminder. The other ten tasks address narrower HSI timing effects; this analysis focuses on the three RTC tasks because the legal LSI range and alarm fault probes apply to them. The models could not build, run, or revise tests from feedback. Functional review, build, and five nominal passes defined eligibility; timing-bound adequacy was measured separately. Across all 13 tasks, 330/390 passed functional review and 321/390 passed nominal screening. The RTC analysis covers 90 attempts and 66 eligible suites.

The frozen callback and pending prompts state these *Required observable checks*, respectively:

> "Neither callback is observed before the alarm time. Both callbacks fire at the configured RTC time and report the expected alarm IDs. The test checks callbacks through observable events or protected state and applies a finite, justified bound."
>
> "The test configures and reads back a supported RTC alarm time successfully. The alarm is not observed pending before its configured RTC time. The alarm is observed pending after the RTC reaches its configured time, under a finite, justified observation bound."

Schedule and attribution. Each measured suite ran in a fresh Renode process (an instruction-level embedded simulator) [2] at 100 shared, uniformly sampled integer LSI settings and both endpoints. ELF, source, CPU, SysTick, and synchronization quantum

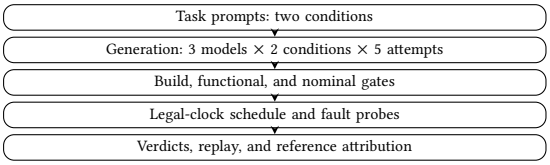


Figure 1: Evaluation pipeline for each RTC task. Generated sources pass eligibility gates before clock and fault challenges.

Table 1: Upstream legal-LSI thresholds and sampled failures.

Case	Boundary	Analytic	Sampled
Callback	24.615 kHz	25.38%	26/100
Counting	22.069 kHz	16.90%	17/100
Pending	24.615 kHz	25.38%	26/100

were fixed. We measured all 36 suites without a scalar clock prediction and a balanced sample of 14 of the 30 predictable suites; 16 remain forecast only. Assertion flips were classified using saved traces, source boundaries, same-input replays, and task-specific references that check premature behavior and wait under a 26-second experimental ceiling. Ambiguous, firmware, simulator, and harness outcomes remain separate. At nominal host settings, all 66 eligible generated suites and three upstream cases faced the oscillator and nonarrival probes; 39 generated and two upstream alarm cases faced the signed alarm offsets. Twenty-seven generated and one upstream counting case were non-activation controls. The task is the primary comparison unit; the registered prompt contrast subtracts base-prompt rates from clock-range-prompt rates within model and task, then averages tasks and models equally. Missing verdicts and forecasts are never recoded as observed passes. The fault probes, alarm-offset probes, and all-attempt, paired, and similarity analyses were added after varied-clock runs began; the dated amendment log is in the repository.

4 RESULTS

4.1 Legal-clock flips (RQ1)

The selected human-written tests failed at many legal slow-clock settings, while most measured generated tests held up. A fixed observation window T has an ideal failure boundary $f < 320/T$ kHz. For the 13-second wait in the Introduction, the alarm arrives too late below 24.615 kHz in this ideal model. Its analytic failing fraction over a continuous uniform 17–47 kHz interval is $\max(0, \min(1, (320/T - 17)/30))$. Table 1 compares these fractions with the registered samples. Small differences reflect finite integer-frequency sampling; counting uses an approximate last-poll boundary.

Across the 53 measured suites, including upstream controls, 5,300 seeded originals yielded 5,182 passes, 106 assertion failures, and 12 harness errors. All 112 seeded-plus-endpoint assertion failures reproduced in persisted replay fields. References passed 3/3 nominal controls, 6/6 endpoints, and 69/69 distinct failing-setting challenges; source boundaries and traces classify all 112 as brittle test assumptions. The 12 errors stay missing.

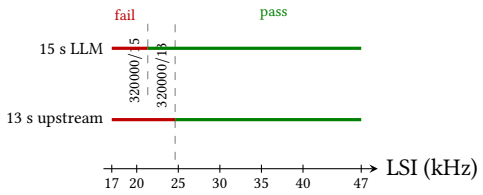


Figure 2: Computed ideal-clock pass/fail steps for a 13-second upstream window and a representative 15-second LLM window. A test fails when $320000/f$ exceeds its window. The registered per-seed forecasts and observed vectors, not this schematic, determine the reported counts.

Table 2: Generated RTC clock outcomes by model and prompt. Base is the base prompt; Ranges adds clock ranges. Failures use evaluable measured seeds.

Model	Prompt	Eligible	Measured	Fail/evaluable
Luna	Base	4	2	0/200
Luna	Ranges	3	2	0/200
Sol	Base	15	11	24/1096
Sol	Ranges	15	13	0/1292
Astra	Base	15	11	13/1100
Astra	Ranges	14	11	0/1100

Table 3: Registered clock-range minus base-prompt hybrid contrasts in percentage points. Missing outcomes are bounded.

Model	Task or summary	Contrast
Sol	Callback	−0.2 to +1.2
Sol	Counting	0
Sol	Pending	−5.4 to −4.4
Sol	Equal-task mean	−1.87 to −1.07
Astra	Callback/counting	0
Astra	Pending	−2.6
Astra	Equal-task mean	−0.87
Luna	Counting only	0
All	Equal-model mean	−0.91 to −0.64

All 37 generated-suite failures came from three base-prompt pending suites: Sol r01 (11), Sol r03 (13), and Astra r05 (13). Their full forecast vectors matched. Measured base-prompt suites had 37 failures in 2,400 seed slots, with four missing; suites prompted with clock ranges had zero in 2,600, with eight missing. At suite level, 3/24 base-prompt suites versus 0/26 clock-range-prompt suites flipped (two-sided Fisher $p = 0.103$). Table 2 keeps measured and forecast-only suites distinct.

Hybrid bounds combine measured verdicts with 16 frozen forecasts for unrun suites and bracket missing outcomes as passes or failures; they are not fully measured rates. Table 3 gives the registered prompt contrasts. Suite-weighted sensitivity and missing-outcome details are in the repository's analysis materials.

4.2 Paired comparisons on the registered settings

Generated suites usually passed the same clock settings that failed their task-matched upstream case. Each measured generated suite

Table 4: Paired clock verdicts on identical evaluable settings. Discordances count upstream failures with generated passes (U only) and the reverse (LLM only).

Task	Suites $p < 0.05$	U only	LLM only
Callback	19/19	492	0
Pending	20/20	480	0
Counting	11/11	187	0

was compared with its matched upstream case on identical evaluable frequencies. Table 4 summarizes exact per-suite McNemar tests and paired discordances; seven callback and five pending seed pairs with missing generated verdicts were omitted. The 50 suites reuse the same settings and share only three task families, so these tests describe this schedule rather than 50 independent replications.

4.3 Fault detection (RQ2)

Surviving legal clock speeds did not mean a test would detect a faulty alarm or oscillator. The ideal legal ten-second alarm-arrival window is 6.809–18.824 host seconds. The 64-kHz oscillator reaches the target at 5.000 s; the 10-kHz oscillator reaches it at 32.000 s. The 13-second wait from the Introduction detects the slow fault because the alarm arrives well after the wait; a much longer wait may miss it. The fast-clock probe needs an independent time reference, whereas the alarm-offset probes change the compare relative to the nominal RTC calendar. Every selected upstream and generated suite survived the fast oscillator. Slow-fault and nonarrival detections are in Table 5; the one missing Luna nonarrival original is unscored. Over-padding mattered: 0/12 suites with a scalar budget above twice the legal slow bound detected the slow fault, versus 18/18 at or below twice the bound. By wait style, slow-fault detections were 39/53 for fixed-sleep-only tests, 7/9 for event-wait-only tests, and 3/4 for mixed tests (tabulations in the repository's analysis directory).

At nominal clock, the upstream callback detected neither signed one-RTC-second alarm offset, whereas generated callback suites detected 18/18 evaluable early and 14/18 late. The pending task is readback-heavy by specification: all 20 generated pending suites detected both signs; upstream pending detected late, while its early original had no verdict. All 56 counting-control runs passed without alarm programming. Of 73 activated detections, 72 first failed a programmed-compare or calendar-relative check; one composite flag had an unresolved immediate cause. The four late-offset survivors were Sol base-prompt callback r01 and r02, whose final assertions accept RTC second 5 *or* 6, and Sol clock-range-prompt callback r01 and r05, which lack an upper calendar-time bound. r05 additionally allows a 60-second deadline. Missing originals are never replaced by diagnostic reruns.

4.4 Prediction and generation yield (RQ3)

The simple clock model predicted observed failures exactly when a test had one clear waiting deadline. A clock-only threshold applied to 30/66 eligible RTC suites. Eleven of 14 sampled predictable suites had zero-failure forecasts with budgets above the legal slow-arrival bound. The three nontrivial forecasts matched exact 100-setting vectors, including 37/37 failures; agreement across the full sample was 1,400/1,400. The other 16 predictable suites remain unrun forecasts. The frozen forecast ledger in the repository's analysis directory records their budgets.

Across all 90 RTC attempts, 15 failed to build, eight failed the functional gate, and one failed nominal screening. Thirty-nine attempts produced a confirmed clock-robust test; 29 also detected the slow and nonarrival faults. A stricter source-coded criterion requiring a known, non-over-padded clock budget admitted 5/90. Table 7 preserves the condition-level denominator. Luna had no eligible alarm suite; its counting suites prompted with clock ranges used 60-second windows and all missed the slow fault. Sol had two clock-flipping pending suites and weaker slow-fault detection with clock ranges; Astra had one flipping pending suite and 14/14 slow-fault detections when prompted with clock ranges. Between-model Fisher tests are exploratory because the three tasks share a driver and emulator.

4.5 Reference timing and upstream source patterns

At 28 measured callback-reference frequencies, ztest duration minus $320000/f$ ranged from -0.001 to $+0.127$ s; this supports Renode's internal timing, not silicon fidelity. Among 33 scanned upstream RTC, counter, and timer files, none explicitly measured interrupt-to-callback latency against a registered bound.

4.6 Supporting source review (RQ4)

Many suites that survived clock variation still assumed something about callback delivery or ordering. A single unblinded automated review flagged latency or ordering assumptions in 28/66 eligible RTC suites; 24 were unknown and 14 had no flagged pattern. Among 47 measured suites without clock-induced assertion failures, 21 had a definite latency premise and 23 were unknown. The three selected upstream RTC files were marked as clock-bound shortfalls in the 33-file scan.

4.7 Source similarity to upstream

A rule-based audit compared all 90 generated RTC sources with matched upstream tests across assertion types, fault profiles, API calls, and token overlap. The maximum pairwise five-token Jaccard score was 0.131, flagging no near-copy under this metric but not ruling out smaller reused fragments or training exposure.

5 DISCUSSION AND THREATS TO VALIDITY

The selected upstream callback checks absence after one fixed sleep and occurrence after another. By contrast, an Astra generated callback captures RTC calendar time and asserts equality with the requested target second. Figure 3 shows the source excerpts. The former accepts either alarm offset; the latter detects both. This is an illustrative, task-matched contrast: the generated prompt asked for configured-time firing, while Zephyr's upstream tests run across many boards and drivers where a lenient window may support portability. The pending prompt expressly required readback, so its detections mainly show instruction-following. Still, four generated callback suites missed the late offset despite the explicit instruction. A stronger oracle would wait for an event

Table 5: Out-of-spec oscillator and nonarrival kills at nominal host timing, with descriptive 95% Wilson intervals. One Luna base-prompt nonarrival original is missing.

Model	Prompt	Fast oscillator	Slow oscillator	Nonarrival
Upstream	n/a	0/3 [0, 56.1]%	3/3 [43.9, 100]%	3/3 [43.9, 100]%
Luna	Base	0/4 [0, 49.0]%	4/4 [51.0, 100]%	3/3 [43.9, 100]% +1 missing
Luna	Ranges	0/3 [0, 56.1]%	0/3 [0, 56.1]%	3/3 [43.9, 100]%
Sol	Base	0/15 [0, 20.4]%	13/15 [62.1, 96.3]%	15/15 [79.6, 100]%
Sol	Ranges	0/15 [0, 20.4]%	9/15 [35.7, 80.2]%	15/15 [79.6, 100]%
Astra	Base	0/15 [0, 20.4]%	9/15 [35.7, 80.2]%	15/15 [79.6, 100]%
Astra	Ranges	0/14 [0, 21.5]%	14/14 [78.5, 100]%	14/14 [78.5, 100]%
All LLM	both	0/66 [0, 5.5]%	49/66 [62.6, 83.3]%	65/65 [94.4, 100]% +1 missing

Table 6: Alarm-offset kills by matched task at nominal clock, with descriptive 95% Wilson intervals for generated suites. Missing originals are excluded.

Case	−1 RTC s	+1 RTC s
Upstream callback	0/1	0/1
Generated callback	18/18 [82.4, 100]%	14/18 [54.8, 91.0]%
Upstream pending	missing	1/1
Generated pending	20/20 [83.9, 100]%	20/20 [83.9, 100]%

Table 7: All-attempt RTC yield. Each row contains 15 generation attempts. Robust means 100 observed legal-clock passes without missing verdicts; +2 also kills slow and nonarrival faults. Base is the base prompt; Ranges adds clock ranges.

Model	Prompt	Working	Robust	Robust+2
Luna	Base	4/15	2/15	2/15
Luna	Ranges	3/15	2/15	0/15
Sol	Base	15/15	6/15	5/15
Sol	Ranges	15/15	8/15	3/15
Astra	Base	15/15	10/15	8/15
Astra	Ranges	14/15	11/15	11/15

under a justified worst-case ceiling, check earliest legal arrival against an independent clock, and verify the RTC calendar target.

The baseline has only three purposively selected, correlated cases and one board, driver, and Renode model. Clock inputs do not bound delivery latency; our one-tick injection failed reference feasibility, so dynamic latency results remain future work. References and replays show internal consistency, not silicon fidelity. Sixteen generated suites have forecasts rather than measured legal-clock outcomes; 12 RTC original verdicts are missing. Wilson intervals over seeds and Fisher/McNemar tests over related suites do not supply independent task replication. The prompt with clock ranges also included a generic reminder that tolerance does not bound callback delivery, so numeric-range effects cannot be isolated. A looser-spec control is also needed to separate requested precision from model-chosen precision. Source and padding labels have one unblinded automated reviewer, without inter-rater agreement; model backend version snapshots and training-data overlap with upstream tests are unavailable.

6 RELATED WORK

Rerun studies link asynchronous waits to flakiness [11]; FlakeRake uses targeted pauses [12], and Strandberg et al. study embedded test assumptions [15]. Berndt et al. examine LLM-generated flaky tests [5]. Mutation testing and test-oracle surveys frame the distinction between fault detection and correct judgments [4, 10]. ChatUniTest, CoverUp, and MuTAP generate tests using LLMs [1, 6, 7]; *All Smoke, No Alarm* studies oracle signals in agent-authored tests [3]. EmbedAgent and EmbC-Test address embedded development and testing [9, 18]. Interrupt-analysis work includes Regehr's schedule testing, SDRacer, AIM, and AidFuzzer [8, 13, 16, 17]. Our study holds test binaries fixed while varying a documented peripheral clock and scores fault detection separately.

7 CONCLUSION AND FUTURE WORK

On these selected RTC tasks, legal clock variation exposed upstream timing windows and three generated pending suites. Most eligible generated suites survived, but over-padding and missing upper calendar checks weakened some fault probes. Thus legal-clock survival alone does not establish a strong alarm oracle. For RTC alarms, bound waits by the worst-case legal clock with a justified delivery margin, check premature arrival against an independent time reference, and assert the requested RTC calendar target rather than eventual arrival alone.

Planned follow-up. A frozen but deferred callback design adds looser upstream-style and reminder-only prompts alongside fresh base and numeric-range controls, five suites per model and condition. A same-spec human baseline packet is prepared, without participants or results. These controls would separate requested from model-chosen precision and numeric-range information from a generic timing reminder. Physical-board timing, delivery-latency variation, and independently coded source review remain future work.

AI USE

LLMs generated the evaluated test suites. Automated tooling assisted experimental execution and record auditing under the authors' direction; the authors reviewed the evidence and take responsibility for the claims.

Upstream callback, `test_alarm_callback.c`, lines 120, 126–127, 132, 140–141:

```
k_sleep(K_SECONDS(RTC_TEST_ALARM_TEST_NOT_CALLED_DELAY));
zassert_equal(callback_called_mask_status_odd, 0,
              "Alarm callback called prematurely");
k_sleep(K_SECONDS(RTC_TEST_ALARM_TEST_CALLED_DELAY));
zassert_equal(callback_called_status, true,
              "Alarm %d callback should have been called", j);
```

LLM-generated Astra base-prompt callback r01, `test.c`, lines 68 and 136–140:

```
o->time_result = rtc_get_time(dev, &o->time);
zassert_true(calendar_compare(&o->time, &target) >= 0,
             "Alarm %u fired prematurely", (unsigned int)i);
zassert_equal(calendar_compare(&o->time, &target), 0,
              "Alarm %u fired outside the target RTC second",
              (unsigned int)i);
```

Figure 3: Literal, noncontiguous excerpts from the pinned upstream callback case and saved Astra base-prompt callback r01. The first checks a fixed window; the second checks the calendar captured at callback against the target.

CODE AND DATA AVAILABILITY

Code, scripts, frozen prompts, and the dated amendment log: https://github.com/pkrish5/zephyr-rtc-clock-robustness. Generated tests are in `generated/`; run records are at https://github.com/pkrish5/zephyr-rtc-clock-robustness/tree/main/results.

REFERENCES

[1] Juan Altmayer Pizzorno and Emery D. Berger. 2025. CoverUp: Effective High Coverage Test Generation for Python. *Proceedings of the ACM on Software Engineering* 2, FSE, 2897–2919. https://doi.org/10.1145/3729398

[2] Antmicro. n.d. Renode time framework documentation. Renode documentation (accessed 2026-10-13). https://renode.readthedocs.io/en/latest/advanced/time_framework.html

[3] Dipayan Banik, Kowshik Chowdhury, and Shazibul Islam Shamim. 2026. All Smoke, No Alarm: Oracle Signals in Agent-Authored Test Code. arXiv:2606.18168. https://arxiv.org/pdf/2606.18168

[4] Earl T. Barr, Mark Harman, Phil McMinn, Muzammil Shahbaz, and Shin Yoo. 2015. The Oracle Problem in Software Testing: A Survey. *IEEE Transactions on Software Engineering* 41, 5, 507–525. https://doi.org/10.1109/TSE.2014.2372785

[5] Alexander Berndt, Thomas Bach, Rainer Gemulla, Marcus Kessel, and Sebastian Baltes. 2026. On the Flakiness of LLM-Generated Tests for Industrial and Open-Source Database Management Systems. In *Proceedings of the 48th International Conference on Software Engineering: Software Engineering in Practice*, 819–830. https://doi.org/10.1145/3786583.3786919

[6] Yinghao Chen, Zehao Hu, Chen Zhi, Junxiao Han, Shuiguang Deng, and Jianwei Yin. 2024. ChatUniTest: A Framework for LLM-Based Test Generation. arXiv:2305.04764. https://arxiv.org/abs/2305.04764

[7] Arghavan Moradi Dakhel, Amin Nikanjam, Vahid Majdinasab, Foutse Khomh, and Michel C. Desmarais. 2024. Effective Test Generation Using Pre-trained Large Language Models and Mutation Testing. *Information and Software Technology* 171, 107468. https://doi.org/10.1016/j.infsof.2024.107468

[8] Bo Feng, Meng Luo, Changming Liu, Long Lu, and Engin Kirda. 2024. AIM: Automatic Interrupt Modeling for Dynamic Firmware Analysis. *IEEE Transactions on Dependable and Secure Computing* 21, 4, 3866–3882. https://doi.org/10.1109/TDSC.2023.3339569

[9] Maximilian Harnot, Sebastian Komarnicki, Michal Polok, and Timo Oksanen. 2026. EmbC-Test: How to Speed Up Embedded Software Testing Using LLMs and RAG. arXiv:2603.09497. https://arxiv.org/pdf/2603.09497

[10] Yue Jia and Mark Harman. 2011. An Analysis and Survey of the Development of Mutation Testing. *IEEE Transactions on Software Engineering* 37, 5, 649–678. https://doi.org/10.1109/TSE.2010.62

[11] Qingzhou Luo, Farah Hariri, Lamyaa Eloussi, and Darko Marinov. 2014. An Empirical Analysis of Flaky Tests. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering*, 643–653. https://doi.org/10.1145/2635868.2635920

[12] Shanto Rahman, Aaron Massey, Wing Lam, August Shi, and Jonathan Bell. 2024. Automatically Reproducing Timing-Dependent Flaky-Test Failures. In *2024 IEEE Conference on Software Testing, Verification and Validation (ICST)*, 269–280. https://doi.org/10.1109/ICST60714.2024.00032

[13] John Regehr. 2005. Random Testing of Interrupt-Driven Software. In *Proceedings of the 5th ACM International Conference on Embedded Software*. https://doi.org/10.1145/1086228.1086282

[14] STMicroelectronics. 2026. STM32F405xx, STM32F407xx Datasheet. STMicroelectronics datasheet DS8626 Rev 12. https://www.st.com/resource/en/datasheet/stm32f407vg.pdf

[15] P. E. Strandberg, T. J. Ostrand, E. J. Weyuker, W. Afzal, and D. Sundmark. 2020. Intermittently Failing Tests in the Embedded Systems Domain. In *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis*, 337–348. https://doi.org/10.1145/3395363.3397359

[16] Jianqiang Wang, Qinying Wang, Tobias Scharnowski, Li Shi, Simon Woerner, and Thorsten Holz. 2025. AidFuzzer: Adaptive Interrupt-Driven Firmware Fuzzing via Run-Time State Recognition. In *34th USENIX Security Symposium*, 1769–1786. https://www.usenix.org/conference/usenixsecurity25/presentation/wang-jianqiang

[17] Yu Wang, Linzhang Wang, Tingting Yu, Jianhua Zhao, and Xuandong Li. 2017. Automatic Detection and Validation of Race Conditions in Interrupt-Driven Embedded Software. In *Proceedings of the 26th ACM SIGSOFT International Symposium on Software Testing and Analysis*, 113–124. https://doi.org/10.1145/3092703.3092724

[18] Ruiyang Xu, Jialun Cao, Mingyuan Wu, Wenliang Zhong, Yaojie Lu, Ben He, Xianpei Han, Shing-Chi Cheung, and Le Sun. 2026. EmbedAgent: Benchmarking Large Language Models in Embedded System Development. In *Proceedings of the 48th International Conference on Software Engineering (ICSE 2026)*. https://doi.org/10.1145/3744916.3787788

[19] Zephyr Project. 2025. Zephyr RTOS v4.3.0 (release commit 3568e1b6d5cd). Zephyr Project source revision. https://github.com/zephyrproject-rtos/zephyr/commit/3568e1b6d5cdd51a6b964a2a1d6d29200fea2056